

Système de recommandation agricole

Sommaire

1. [Présentation](#)
2. [Organisation](#)
3. [Fonctionnalités](#)
4. [Prérequis](#)
5. [Installation](#)
6. [Analyse exploratoire des fichiers sources](#)
7. [Enrichissement du dataset](#)
8. [Feature engineering sur le dataset enrichi](#)
9. [Feature engineering sur le dataset brut](#)
10. [Modélisation](#)
11. [Interprétabilité du modèle feature importance et SHAP values](#)
12. [Pipeline CI CD](#)
13. [Tests et validation des données](#)
14. [Conclusion](#)
15. [Perspectives](#)

Présentation

Ce projet a pour but de réaliser une application web simple et intuitive pour aider les agriculteurs à prendre de meilleures décisions. L'application aura deux fonctions au sein de la même interface :

1. Fonction de prédiction : Permettre à un utilisateur de sélectionner une culture spécifique, de renseigner les conditions de sa parcelle (température, usage de pesticides, etc.) et d'obtenir une estimation chiffrée du rendement attendu.
2. Fonction de recommandation : L'utilisateur renseigne uniquement les conditions de sa parcelle, et l'application lui recommande la culture la plus rentable en simulant le rendement pour toutes les cultures possibles et en affichant un classement.

Nous avons eu à disposition deux datasets :

- Agriculture Crop Yield.zip
 - Fichier crop_yield.csv
- Crop Yield Prediction Dataset.zip
 - pesticides.csv
 - rainfall.csv
 - temp.csv
 - yield_df.csv
 - yield.csv

Analyse exploratoire des fichiers sources

Nous avons eu à disposition deux datasets :

- Agriculture Crop Yield.zip
 - Fichier crop_yield.csv
- Crop Yield Prediction Dataset.zip
 - pesticides.csv
 - rainfall.csv

- temp.csv
- yield_df.csv
- yield.csv

La première chose réalisée a été l'analyse exploratoire de l'ensemble des fichiers.

Fichier crop_yield.csv

Description

On retrouve des informations importantes :

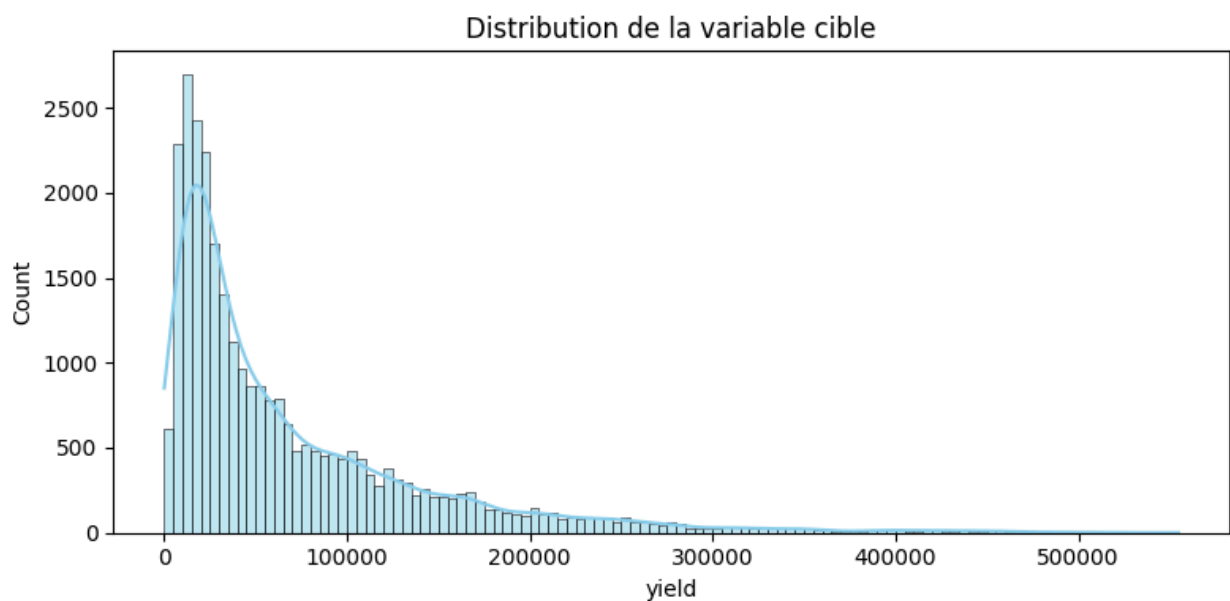
- Type de culture : riz, coton etc
- Grande région : west, south etc
- Conditions climatiques :
 - précipitation
 - température
 - nombre de jours de récolte
 - conditions météorologiques (cloudy, rainy etc)
- Type d'utilisation agricole :
 - utilisation de fertilisant
 - utilisation d'irrigation
- **Variable cible** : le rendement en tonne par hectare -> variable cible
- On retrouve 10 colonnes pour 1 million de lignes
- Le fichier semble avoir des données synthétiques
- Valeur unique par variable catégorielle :

```

Nombre de valeur unique : Soil_Type: 6
Nombre de valeur unique : Crop: 6
Nombre de valeur unique : Fertilizer_Used: 2
Nombre de valeur unique : Irrigation_Used: 2
Nombre de valeur unique : Weather_Condition: 3

```

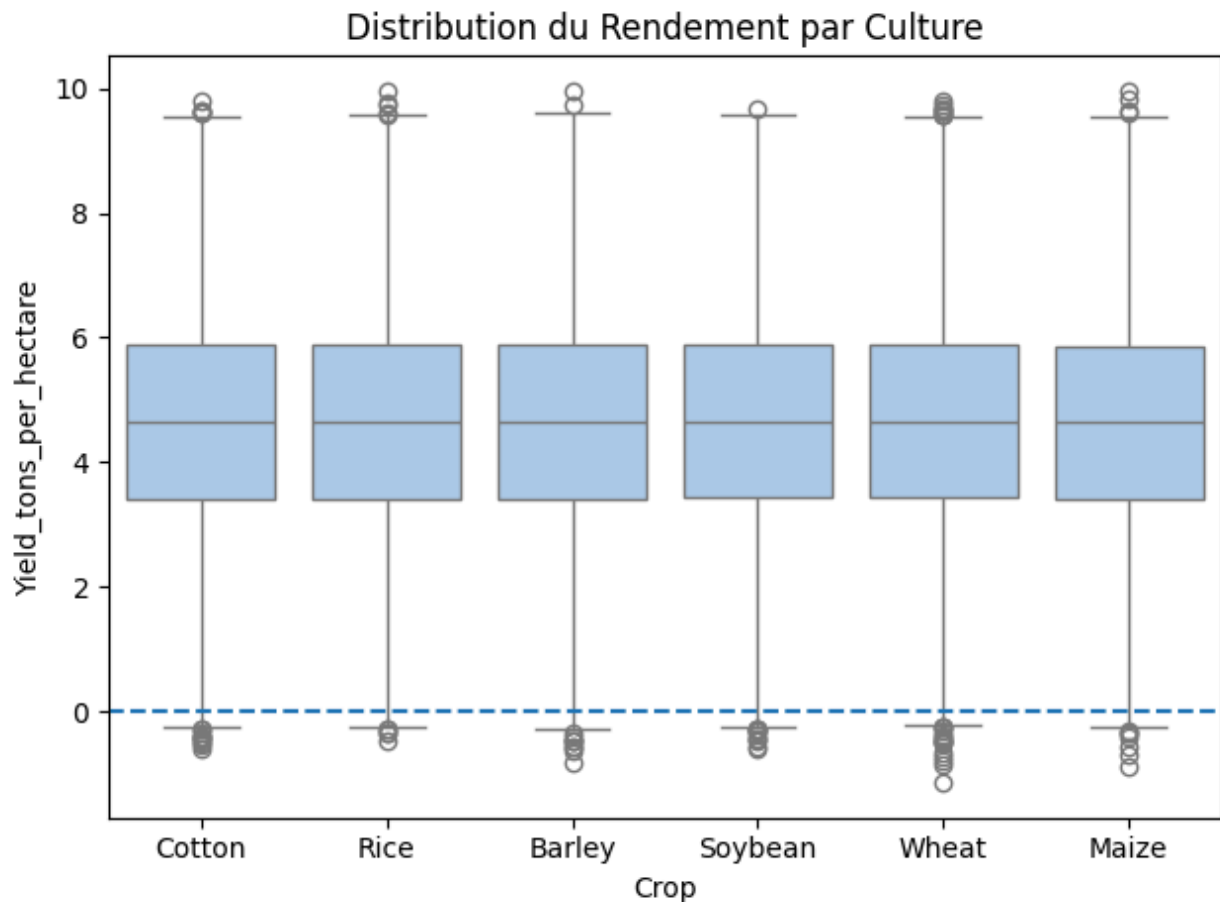
La distribution de notre variable cible :



Notre distribution ressemble à une cloche, donc à une distribution normale même si on ne peut pas la considérer comme telle car on voit quelques petits dépassements.

Nettoyage

- Boxplot pour analyser le type de culture par rapport au rendement.



- On retrouve quelques valeurs aberrantes sur notre variable cible. On peut voir des rendements négatifs donc nous avons décidé de les supprimer car cela concerne uniquement 231 lignes sur les 1 million.
- Les valeurs "extrêmes" supérieures ne constituent pas d'anomalies statistiques mais une réalité du dataset.
- Afin de préparer le fichier pour l'ACP, on décide de mettre les variables **Fertilizer_Used** et **Irrigation_Used** en numérique (0 ou 1 car les valeurs étaient True or False).

Analyse des composantes principales

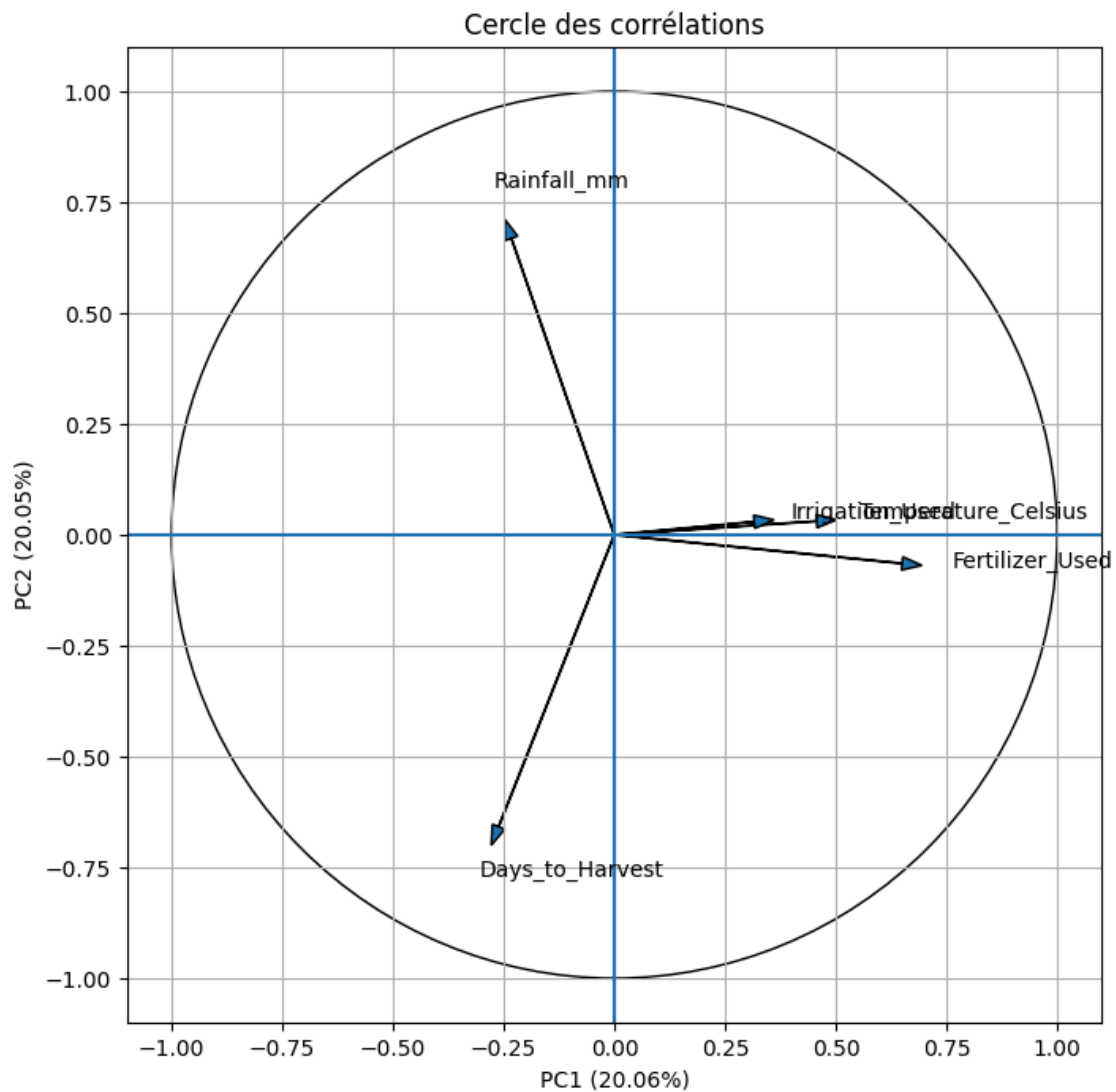
Étant donnée que nous avons beaucoup de données dans ce fichier, nous allons utiliser la méthode ACP pour nous permettre d'aller un peu plus loin dans l'analyse exploratoire.

- Réduction des dimensions :

Nos variables sont indépendantes, ce qui nous ne permet pas de réaliser une bonne réduction des dimensions. Nous avons étudié avec 5 composantes principales, elles sont autour des 20% pour chaque.

- Identification des variables clés :
 - **Même si l'ACP ne réduit pas beaucoup la dimension, elle reste très utile pour comprendre les relations.**
 - PC1 :
 - conditions/intensité de croissance plus fortes" vs "cycle plus long
 - PC2 :
 - PC2 est surtout un axe lié à la pluie, opposé à un bloc de variables agricoles/cycle
 - PC3 :

- irrigation vs température
- PC4 :
 - fertilisation et autres conditions de croissance
- PC5 :
 - cycle plus long et environnement plus humide
- Cercle de corrélation :



- Les variables alignées, elles pointent dans la même direction :
 - Fertilizer_Used
 - Temperature_Celsius
 - Irrigation_Used
- Variable opposée : Days_to_Harvest est négativement corrélée avec celles du dessus
 - plus d'engrais / température / irrigation = moins de jours jusqu'à récolte
- Variable isolée :
 - Rainfall : Rainfall est faiblement corrélée aux autres variables

- 3 dimensions réelles : intensité agricole / durée de culture / conditions climatiques

Proxy

Un proxy est une variable qui remplace quelque chose qu'il ne peut pas être mesuré directement.

Variable, proxy de :

- Irrigation_Used : gestion de l'eau
- Fertilizer_Used : fertilité du sol

Les variables utilisées dans le modèle sont des proxies permettant d'approcher des phénomènes agronomiques complexes, tels que la disponibilité en eau ou la fertilité du sol.

On réalise alors que nous allons se servir des éléments que nous avons ici pour enrichir notre futur fichier pour la modélisation. Voici les éléments que nous chercherons à ajouter :

- Irrigation_Used
- Fertilizer_Used
- Weather_Condition
- Soil_Type

2. Fichiers présents dans Crop Yield Prediction Dataset

Fichier pesticides.csv

On retrouve dans ce fichier l'utilisation en tonne de pesticide par pays et par année.

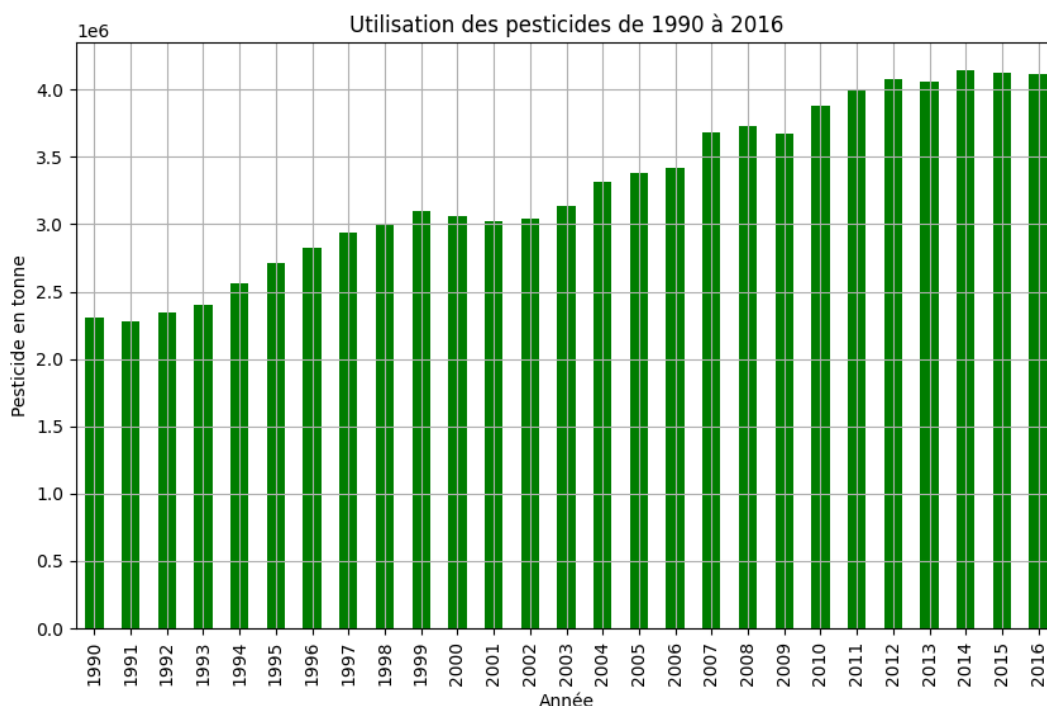
Après analyse du fichier :

- Pas de doublons
- Pas de valeurs manquantes
- Présence de valeurs extrêmes

La présence de valeur extrême est expliquée par un gros écart entre les gros utilisateurs de pesticide et les petits utilisateurs. **Le top 5 des plus gros utilisateurs de pesticide représente 66% de la valeur globale.**

On a analysé les pays en "écart", rien ne semble anormal, ce sont des pays qui peuvent être des gros utilisateurs.

Évolution de l'utilisation des pesticides dans le temps



- Depuis les années 2000 jusqu'en 2006, on remarque une croissance de l'utilisation des pesticides de manière globale.
- Une petite en chute en 2009 avec une légère progression ensuite mais depuis 2011, l'utilisation reste constante.

Fichier rainfall.csv

On retrouve dans ce fichier les précipitations en mm par pays et par année.

Après analyse du fichier :

- Pas de doublons
- Plus de 11% de valeurs manquantes dans la variable des précipitations.
- Identification de 25 pays sans valeur, sans aucune valeur par année.

Liste des pays avec des valeurs nulles :

```
<StringArray>
```

```
[
    'American Samoa',      'Aruba',
    'Bermuda',      'British Virgin Islands',
    'Cayman Islands',      'Channel Islands',
    'Curacao',      'Faroe Islands',
    'French Polynesia',      'Gibraltar',
    'Greenland',      'Guam',
    'Hong Kong SAR, China',      'Isle of Man',
    'Kosovo',      'Macao SAR, China',
    'Monaco',      'New Caledonia',
    'Northern Mariana Islands',      'San Marino',
    'Sint Maarten (Dutch part)',      'St. Martin (French part)',
    'Tonga',      'Turks and Caicos Islands',
    'Virgin Islands (U.S.)']
```

```
Length: 25, dtype: str
```

```
=====
```

Liste des pays avec au moins une valeur non nulle :

```
Area  Year  average_rain_fall_mm_per_year
4061  Monaco  1985                      ..
```

Fichier temp.csv

On retrouve dans ce fichier les températures moyennes par pays et par année.

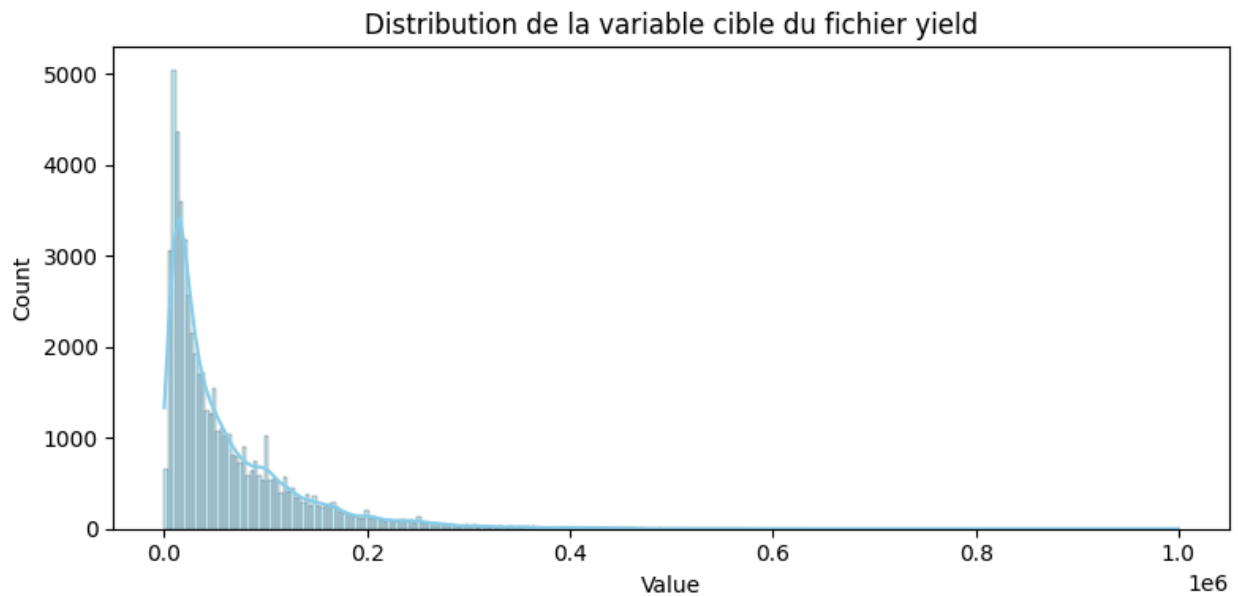
Après analyse du fichier :

- Doublons trouvés : il y a une répétition de plusieurs années par pays.
- 3% de données manquantes pour la variable température.
- On retrouve des dates très anciennes, à partir de 1743.
- Les valeurs de la température vont de -14 à 30 degrés.

Fichier yield.csv

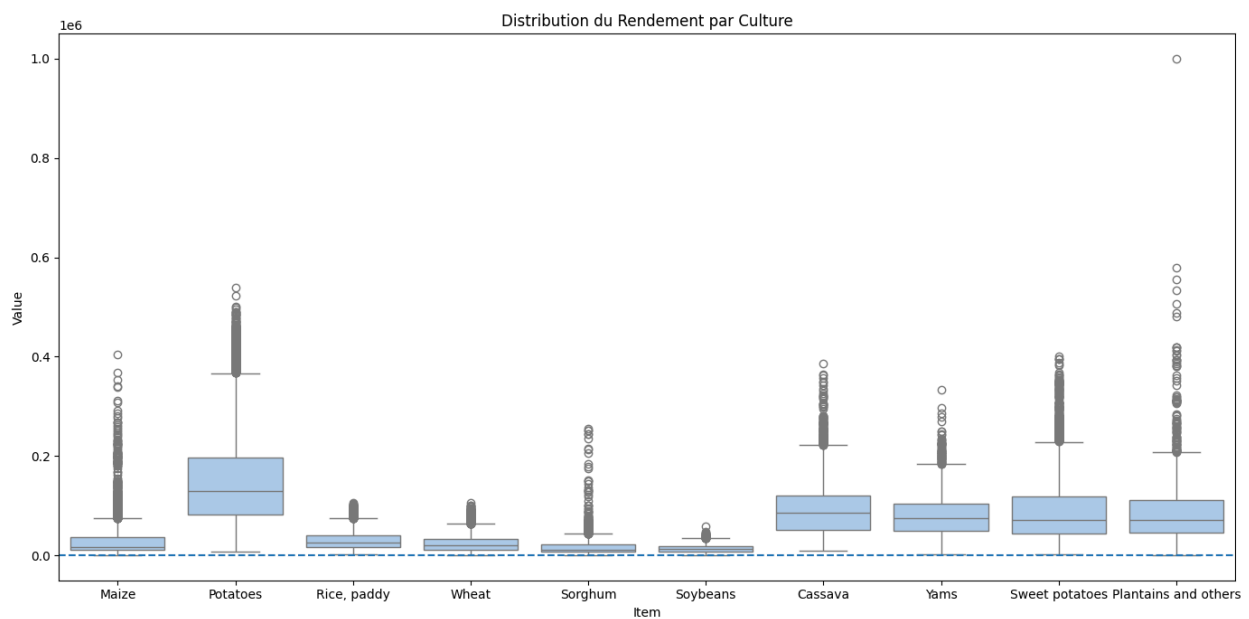
On retrouve dans ce fichier un dataset central par pays, par année, par type de culture ainsi que les valeurs de rendement. Ce fichier est central et sera donc le fichier à consolider avec les fichiers des conditions climatiques.

Distribution de la variable de rendement :



On voit un étalement vers la gauche. Les données se suivent pas une distribution quasi normale comme le premier dataset. Le fichier est + déséquilibré.

Boxplot par type de culture et rendement :



- On peut voir quelques valeurs à 0. Difficile de savoir si on les supprime ou non car théoriquement un rendement à 0 est possible si les conditions ne sont pas réunies pour une production. Mais certaines valeurs ressemblantes ont des données de rendement. Nous avons seulement 8 valeurs à 0 alors on peut les supprimer.
- On voit une valeur énorme pour Plantains and more
 - La valeur extrême date de 1964, nous irons pas dans des dates aussi antérieures donc elle sera enlevée par défaut par la suite.
- On voit un fort déséquilibre entre les types de culture. On peut noter que la catégorie Potatoes semble avoir des rendements plus élevés que les autres. Ce sont des valeurs extrêmes mais pas aberrantes car elles sont réelles alors nous devons les laisser.

Fichier yield_df.csv

On retrouve dans ce fichier un dataset déjà consolidé entre yield et les 3 fichiers concernant les conditions météo.

Après analyse du fichier :

- Difficulté de savoir comment a été traité le fichier

- Pas de valeurs manquantes
- Doublons trouvés sur les températures notamment.

Pour une meilleure analyse, nous allons consolider un nouveau fichier `yield_df` car nous ne savons pas comment ce fichier a été construit.

Nouveau fichier consolidé :

Avant de pouvoir étudier correctement les relations entre les différentes conditions météorologiques, nous devons consolider les fichiers suivants :

- `yield.csv`
- `pesticides.csv`
- `rainfall.csv`
- `temp.csv`

Le fichier central est `yield`, nous allons donc faire des jointures sur ce fichier. Il y a plusieurs problématiques :

- Pas exactement les mêmes noms pour tous les pays
- Un nombre d'année différent entre les fichiers
- Impact potentiel, un nombre de NaN conséquent pour certains pays et certaines années.

Choix méthodologiques :

- Harmonisation des noms de pays pour assurer la compatibilité entre les différentes sources
- Restriction à la période commune (1990–2016) afin d'éviter des valeurs manquantes excessives, notamment sur les données de température et avoir des années récentes
- Fusion progressive des datasets en conservant `yield.csv` comme base principale
- Ajout de la région par pays afin d'avoir un détail plus fin.
- Création de variables indicatrices (région/item/condition climatique) de valeurs manquantes pour conserver l'information liée à l'absence de données --> utilisation des quartiles
- Imputation des valeurs manquantes en fonction des "bins" afin de ne pas imputer des valeurs globales qui pourraient fausser les données
- Cela va nous permettre de limiter la suppression des données

1. Harmonisation du nom des pays

- Extraction de chaque pays uniques dans les 4 fichiers.
- Envoie des pays à un LLM pour lui demander plusieurs noms possibles par pays
- Mapping des fichiers avec un nom de pays standardisé
- Identification d'un doublon supplémentaire grâce à l'harmonisation
- **TOTAL : 44 pays uniques ont été harmonisés sur l'ensemble du projet. Exemple pour un fichier :**

```
--- Rapport pour Yield ---
Nombre total de pays uniques : 212
Nombre de pays modifiés : 28
Exemples de modifications :
  - The former Yugoslav Republic of Macedonia -> North Macedonia
  - Occupied Palestinian Territory -> Palestine
  - Saint Lucia -> St. Lucia
  - China, Taiwan Province of -> Taiwan
  - Cabo Verde -> Cape Verde
```

2. Harmonisation du nom des variables et restriction de la temporalité

- Ajout de nom de variable cohérent
- Temporalité commune : 1990 -> 2016
 - Avoir en tête que le fichier `temp` s'arrêtait en 2013

3. Gestion des doublons sur le fichier temp

- Regroupement des pays par année et par pays en effectuant la médiane de la température

4. Fusion des datasets à partir de yield

- Merge des fichiers un à un
- Passage à 7 colonnes et 29 151 lignes avec des valeurs manquantes pour les conditions climatiques. Valeurs manquantes :
 - avg_temp -> 0.363521
 - pesticides_tonnes -> 0.171452
 - rainfall_mm -> 0.102261

5. Imputation sur les valeurs manquantes

- Ensuite nous voulons améliorer la distinction des pays alors nous avons sélectionné un dataset afin d'ajouter les grandes régions par pays
 - <https://unstats.un.org/unsd/methodology/m49/overview/> : dataset sélectionné
 - Pour cela nous devons faire matcher encore une fois le noms des pays avec le dataset, nous devons modifier "à la main" 37 pays qui ne correspondaient pas
 - Exemple de région :

Sub-region Name	area	
Australia and New Zealand	Australia	189
	New Zealand	108
Central Asia	Kazakhstan	150
	Kyrgyzstan	150
	Tajikistan	149
	Turkmenistan	100
	Uzbekistan	125
Eastern Asia	China	216
	Hong Kong	47
	Japan	189
	Mongolia	54
	North Korea	189
	South Korea	189
	Taiwan	216

- Création de groupe climatique :
 - les variables climatiques divisées en quartiles
 - puis création d'une nouvelle variable avec la région, le type de culture et le numéro de catégorie de chaque variable climatique, exemple : **Central Asia_Wheat_3_2_3**
- Imputation des NaN en fonction de la nouvelle variable

6. Fichier consolidé

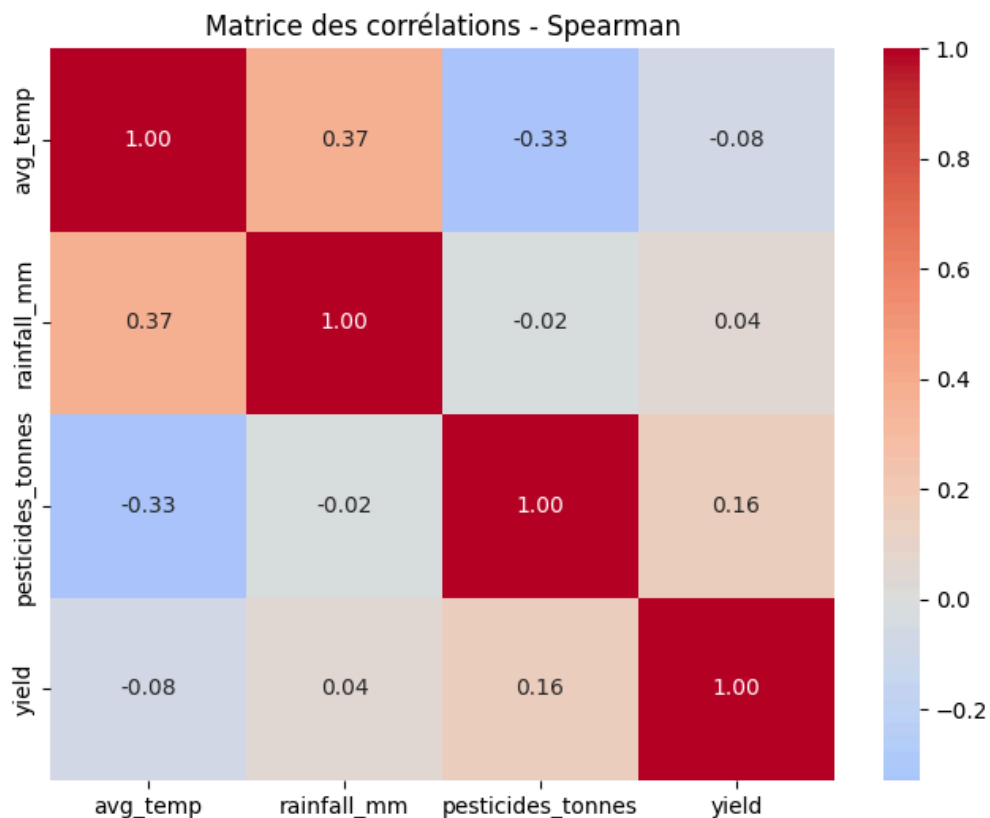
Tableau final

area	region	year	item	avg_temp	rainfall_mm	pesticides_tonnes	yield
Afghanistan	Southern Asia	1990	Maize	15.45	327.0	1594.5	17582
Afghanistan	Southern Asia	1991	Maize	14.57	327.0	1594.5	16800
Afghanistan	Southern Asia	1992	Maize	14.35	327.0	1594.5	15000
Afghanistan	Southern Asia	1993	Maize	14.96	327.0	1594.5	16786
Afghanistan	Southern Asia	1994	Maize	14.94	327.0	1594.5	16667

Cet exemple est pertinent :

- On voit des données de température différente par année pour un même pays : les données étaient ainsi dans le fichier temp
- La région est bien rajoutée
- Pour rainfall les données de base sont répétées par année
- Pour ce pays en particulier il n'y avait pas d'informations pour les pesticides, ces données ont été imputé
- On observe bien un rendement différent par année pour un même pays

Matrice corrélation



- On note une corrélation assez forte entre les températures moyennes et les précipitations
- Avec la valeur cible c'est la valeur des pesticides qui a un lien + fort que les autres
- Relation négative entre les températures et les pesticides ainsi qu'entre les rendements et les températures

Comparaison ACP vs dataset prediction

L'ACP montre que la variance est répartie de manière homogène entre les composantes, ce qui indique l'absence de structure dominante et de fortes corrélations entre les variables explicatives. La matrice de corrélation confirme ce constat, avec des coefficients globalement faibles, traduisant des relations linéaires limitées entre variable, ainsi qu'avec la variable cible.

Enrichissement du dataset

Il n'y a de relation directe entre notre premier dataset (celui pour l'ACP) et les autres datasets.

- Le premier fichier est par grande région et culture
- Le deuxième fichier est par pays, par année puis par culture
- Le rapprochement des données serait très bancal.

Mais nous avons identifié des variables proxy qui pourraient enrichir notre dataset :

- Irrigation_Used
- Fertilizer_Used
- Weather_Condition
- Soil_Type

Le fichier crop_yield semble être un fichier synthétique :

- Pas de corrélation entre les variables explicatives
- Répartition à quasi 50/50 des valeurs pour chaque variable

Plusieurs méthodes d'enrichissement ont été effectuées :

- Modèle de régression logistique afin de prédire les différentes variables sur le nouveau fichier
 - Variables explicative : précipitation et température
 - Après un test, le modèle n'arrive pas à généraliser, les variables explicatives sont indépendantes entre elles. Les données sont "plates" (sans corrélations fortes), le modèle de ML essayait de trouver une règle là où il n'y en a pas
 - Par exemple, la balanced_accuracy était à 0,50 pour le fertilisant et l'irrigation

```
=====
RÉSUMÉ GLOBAL
=====
target      accuracy  balanced_accuracy  f1_macro  log_loss
Fertilizer_Used  0.501995  0.501995          0.501987  0.693143
Irrigation_Used  0.499895  0.499747          0.489134  0.693148
Weather_Condition 0.334300  0.334245          0.333182  1.098607
Soil_Type       0.166765  0.166608          0.142119  1.791780
```

- Méthode de stratification par catégorie afin de faire le rapprochement sur le nouveau fichier en utilisant la valeur la plus fréquente (mode())
 - La mise en place de la stratification était simple (en quartiles)
 - Le problème est l'utilisation de la valeur fréquente. Quand il y a une valeur qui se démarque, cela fonctionne bien. Cependant sur une égalité, le mode choisit aléatoirement
- Méthode de sampling conditionnel afin d'attribuer la distribution de crop_yield par culture/clé climatique puis choix statistique de la valeur avec une fonction random
 - Méthode retenue

Méthode du sampling conditionnel

- Standardisation des noms des types de culture entre les fichiers
- Autre problématique : nous avons uniquement 4 cultures en communs entre les 2 fichiers

Méthode

1. Construction des groupes

Les données sont segmentées selon :

- climate_key (quantiles température + pluie)

- crop (type de culture)

2. Calcul des distributions

Pour chaque variable cible et chaque groupe :

```
P(valeur | climate_key, crop)
```

Exemple :

```
Maize + T2_R4 → Fertilizer_Used :
True  = 70%
False = 30%
```

3. Imputation par sampling

Pour chaque valeur manquante :

- Récupération de la distribution du groupe
- Tirage aléatoire pondéré :

```
np.random.choice(values, p=probabilities)
```

4. Gestion des cas manquants

- crop
- climate_key
- global

5. Conclusion

Les avantages ici sont :

- le respect des distributions réelles
- évite les biais
- conserve la variabilité des données

Les limites sont :

- que c'est non déterministe
- que la cohérence individuelle n'est pas garantie

La méthode est validée par :

- une comparaison des distributions avant / après enrichissement
- faible écart global
- absence de biais sur les catégories

Exemple de comparaison par nouvelle variable :

```
=== Fertilizer_Used ===
Source (crop_yield):
Fertilizer_Used
False    0.50006
True     0.49994
Name: proportion, dtype: float64

Final (yield_df_final):
Fertilizer_Used
False    0.501252
True     0.498748
Name: proportion, dtype: float64

=== Irrigation_Used ===
Source (crop_yield):
Irrigation_Used
False    0.500509
True     0.499491
Name: proportion, dtype: float64

Final (yield_df_final):
Irrigation_Used
True     0.503482
False    0.496518
Name: proportion, dtype: float64
```

Exemple de comparaison par type de culture :

=== Fertilizer_Used par crop ===

		source	final
	Fertilizer_Used		
Barley	False	0.500741	NaN
	True	0.499259	NaN
Cotton	True	0.500609	NaN
	False	0.499391	NaN
Maize	False	0.501547	0.497063
	True	0.498453	0.502937
Rice	True	0.500384	0.500473
	False	0.499616	0.499527
Soybean	False	0.500219	0.502883
	True	0.499781	0.497117
Wheat	True	0.501155	0.501368
	False	0.498845	0.498632
Cassava	False	NaN	0.508120
	True	NaN	0.491880
Plantains and others	False	NaN	0.506292
	True	NaN	0.493708
Potatoes	True	NaN	0.511170
	False	NaN	0.488830
Sorghum	False	NaN	0.519462
	True	NaN	0.480538

Feature engineering sur le dataset enrichi

Nous avons désormais consolidé notre dataset :

- Une première fois avec 3 variables climatiques puis imputer de manière rigoureuse pour les données manquantes.
- Une deuxième fois en enrichissant le dataset avec des variables approximatives qui devraient aider notre modèle à mieux généraliser

Nous devons maintenant continuer l'amélioration de nos données en ajoutant des données si possible, et en transformant nos données pour être compréhensible pour un modèle.

Nouvelles variables

```

Ratio climatique – indice d'irrigation critique :
yield_df_enriched['irrigation_impact'] = yield_df_enriched['Irrigation_Used'].astype(int) /
(yield_df_enriched['rainfall_mm'] + 1)

Flag de sécheresse (Binaire) :
yield_df_enriched['is_drought'] = (yield_df_enriched['rainfall_mm'] < 200).astype(float)

Déséquilibre intrants/eau (Ecart log) :
yield_df_enriched['input_imbalance'] = np.abs(np.log1p(yield_df_enriched['pesticides_tonnes']) -
np.log1p(yield_df_enriched['rainfall_mm']))

Stress thermique simple :
yield_df_enriched['thermal_stress'] = np.abs(yield_df_enriched['avg_temp'] - 20)

On fixe l'année de référence 2016 pour l'interface future :
year_ref = 2016

On crée un score de maturité technologique. Plus l'écart est grand, plus les semences et machines sont supposées
performantes :
yield_df_enriched['years_from_now'] = year_ref - yield_df_enriched['year']

```

- Pas relation forte entre nos variables mais on voit que l'ajout des nouvelles variables ne sont pas redondantes avec les features d'origine.

Feature engineering sur le dataset brut

Petit rappel de la problématique sur les fichiers de ce projet. L'entreprise nous a donné 2 datasets dont un qui présente selon nous des données synthétiques (le fichier crop_yield).

Nous avons essayé de les rapprocher de la manière la plus propre possible statistiquement mais n'ayant pas la possibilité de les joindre de manière "directe" nous avons utilisé la méthode de sampling conditionnel pour rajouter les éléments qui manquaient dans notre fichier de base.

C'est pour cela que nous avons décidé de tester également le fichier brut sans les éléments rajoutés (nommés ici enrichissement) afin d'utiliser le meilleur dataset possible pour notre projet. Nous les comparerons avec des métriques ML pour choisir.

Pour réaliser cela, nous devons effectuer le même feature engineering sur le fichier brut.

Nouvelles variables

Flag de sécheresse (Binaire) :

```
yield_df_final_conso['is_drought'] = (yield_df_final_conso['rainfall_mm'] < 200).astype(float)
```

Déséquilibre intrants/eau (Ecart log) :

```
yield_df_final_conso['input_imbalance'] = np.abs(np.log1p(yield_df_final_conso['pesticides_tonnes']) -  
np.log1p(yield_df_final_conso['rainfall_mm']))
```

Stress thermique simple :

```
yield_df_final_conso['thermal_stress'] = np.abs(yield_df_final_conso['avg_temp'] - 20)
```

On fixe l'année de référence 2016 pour l'interface future :

```
year_ref = 2016
```

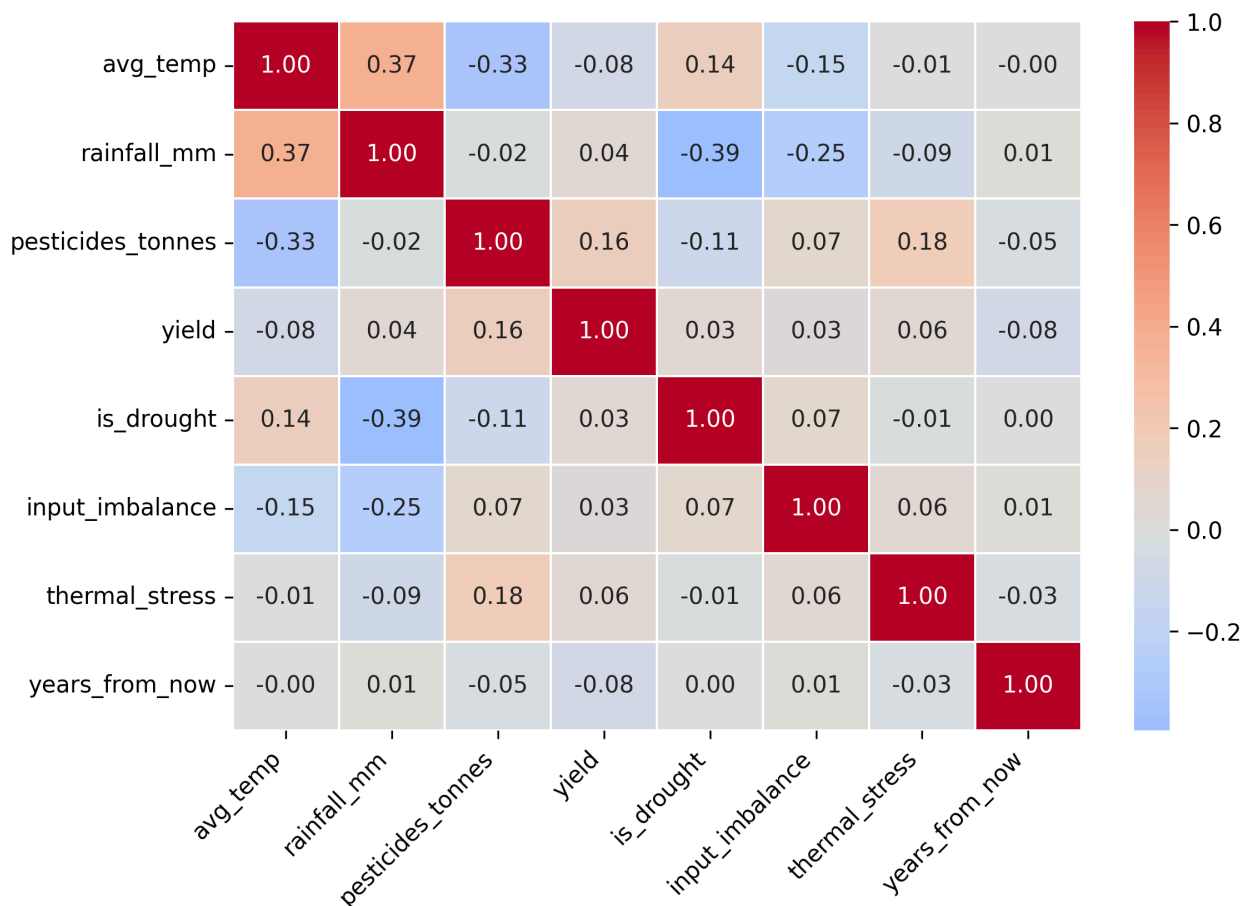
On crée un score de maturité technologique. Plus l'écart est grand, plus les semences et machines sont supposées performantes :

```
yield_df_final_conso['years_from_now'] = year_ref - yield_df_final_conso['year']
```

Sans avoir enrichi notre fichier, on ne peut pas ajouter la variable `irrigation_impact` qui dépend de `Irrigation_Used`.

Matrice des corrélations

Matrice des corrélations - Spearman



Nous n'avons pas ici des variables trop corrélées ensemble, nous n'avons pas à enlever d'autres feature.

Modélisation

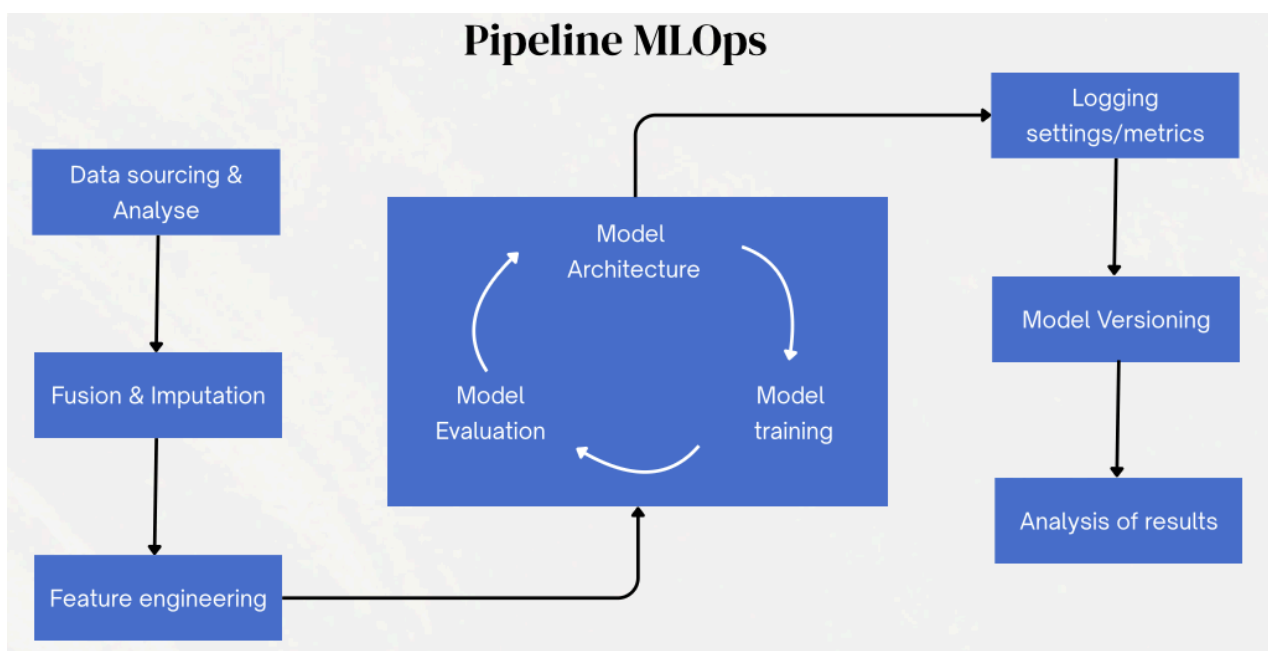
Nous allons passer à l'évaluation de nos 2 fichiers afin de choisir le plus pertinent pour notre modèle. Afin d'être le plus clair possible, nous nommerons les fichiers ainsi :

- **dataset brut** - c'est le fichier consolidé après nettoyage & fusion des conditions climatiques
- **dataset enrichi** - c'est le fichier consolidé après nettoyage & fusion des conditions climatiques avec en + les features rajoutées lors de l'enrichissement des données

Pour le suivi de la modélisation avec des pratiques de MLOps, nous utilisons **MLFlow** pour suivre les expérimentations. L'ensemble des tests ont été réalisés dans le notebook dédié à la modélisation. À l'intérieur on va retrouver plusieurs blocs :

- un suivi MLFlow de 5 modèles sur le dataset brut
- un suivi MLFlow de 5 modèles sur le dataset enrichi
- un suivi MLFlow pour l'optimisation des hyperparamètres du modèle retenu

Schéma MLOps



Choix des modèles

On a testé 5 modèles différents :

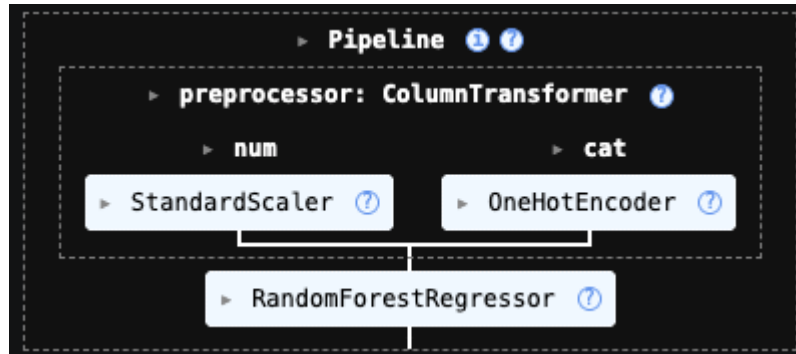
- DummyRegressor
- LinearRegression
- RandomForestRegressor
- XGBRegressor
- LGBMRegressor

Pourquoi ces modèles ?

- D'abord, une baseline minimale avec DummyRegressor. Le but était d'avoir un point de comparaison naïf. Ça permet de vérifier que le pipeline apprend réellement quelque chose et qu'on ne se satisfait pas d'un score "bon en apparence".
- Ensuite, une baseline linéaire avec LinearRegression. Utilisation pour tester l'hypothèse la plus simple, si le rendement s'explique surtout par des relations linéaires entre climat, intrants et culture.
- Ensuite on a testé 3 trois familles d'arbres car le problème mélange :
 - des variables numériques comme avg_temp, rainfall_mm, pesticides_tonnes, thermal_stress etc
 - des variables catégorielles comme region, item, is_drought
 - et potentiellement des relations non linéaires et des interactions fortes entre climat, zone géographique et type de culture.

Choix du pipeline

- Pipeline scikit-learn : le preprocessing et le modèle sont encapsulés ensemble, donc on évite les écarts entre train et test.
- Cross-validation : les scores ne reposent pas sur un seul split.
- `random_state=42` sur les modèles et le KFold : les résultats sont reproductibles.
- MLflow : chaque run conserve paramètres, métriques et version du modèle.
- Le meilleur pipeline est sauvegardé avec joblib, ce qui facilite la réutilisation et l'audit.
- Les analyses SHAP et feature importance renforcent l'explicabilité en plus de la feature importance.
- Schéma du pipeline :



Robustesse du pipeline

- `GridSearchCV(..., error_score="raise")` : si un entraînement échoue, l'erreur remonte immédiatement.
- si MLflow n'est pas disponible, le tracking peut échouer.
- si un fichier CSV ou un artefact `.joblib` manque, le notebook s'arrête.
- les cellules SHAP dépendent du pipeline sauvegardé, donc si le modèle n'est pas exporté avant, cette partie casse.

Métriques des performance des modèles

- Coefficient de détermination (R^2) : mesure la part de variance expliquée par le modèle.
- Erreur quadratique moyenne (RMSE) : mesure l'erreur moyenne en donnant plus de poids aux grosses erreurs.
- Erreur absolue moyenne (MAE) : moyenne des erreurs en valeur absolue
- Erreur relative (MAPE) : erreur en pourcentage
- Erreur économique (`economic_error_usd_ha`) : c'est une métrique métier personnalisée, elle convertit l'erreur de prédiction en perte financière par hectare
 - Nous avons récupéré les prix par tonne au niveau mondial pour chaque type de culture pour l'année de 2016 (dernière année de notre dataset)

Résultats des performances sur le dataset brut

Résumé de la première étape de modélisation

Les expérimentations montrent que les modèles d'arbres surpassent nettement les baselines simples.

Le **RandomForestRegressor** obtient les meilleures performances globales sur le dataset brut, avec un **R^2 test de 0,9436**.

Le dataset enrichi n'a pas permis d'améliorer les performances de manière significative. Le choix final s'est donc porté sur le **dataset brut**, plus simple, plus lisible et tout aussi performant.

Dans le détail, voici ce que nous avons fait

Sur les 2 modèles "simples"

- Sur le DummyRegressor et le LinearRegression les résultats ne sont pas bons, c'était attendu. Le Dummy nous sert comme base pour comparer et nous ne sommes pas face à une distribution linéaire alors LinearRegression ne généralise pas bien. Les modèles linéaires montrent leurs limites sur ce problème, ce qui confirme la présence de relations non linéaires entre les variables explicatives et le rendement.
- DummyRegressor :

```
=== Résultats métriques ===  
CV RMSE : 7604.3730 (± 187.80)  
CV MAPE : 2.6730 (± 0.1885)  
CV R2 : -0.0002 (± 0.0002)  
Test RMSE : 7216.6343  
Test R2 : -0.0013  
Test MAPE : 2.4448  
Test economic_error_usd_ha : 1680.7060
```

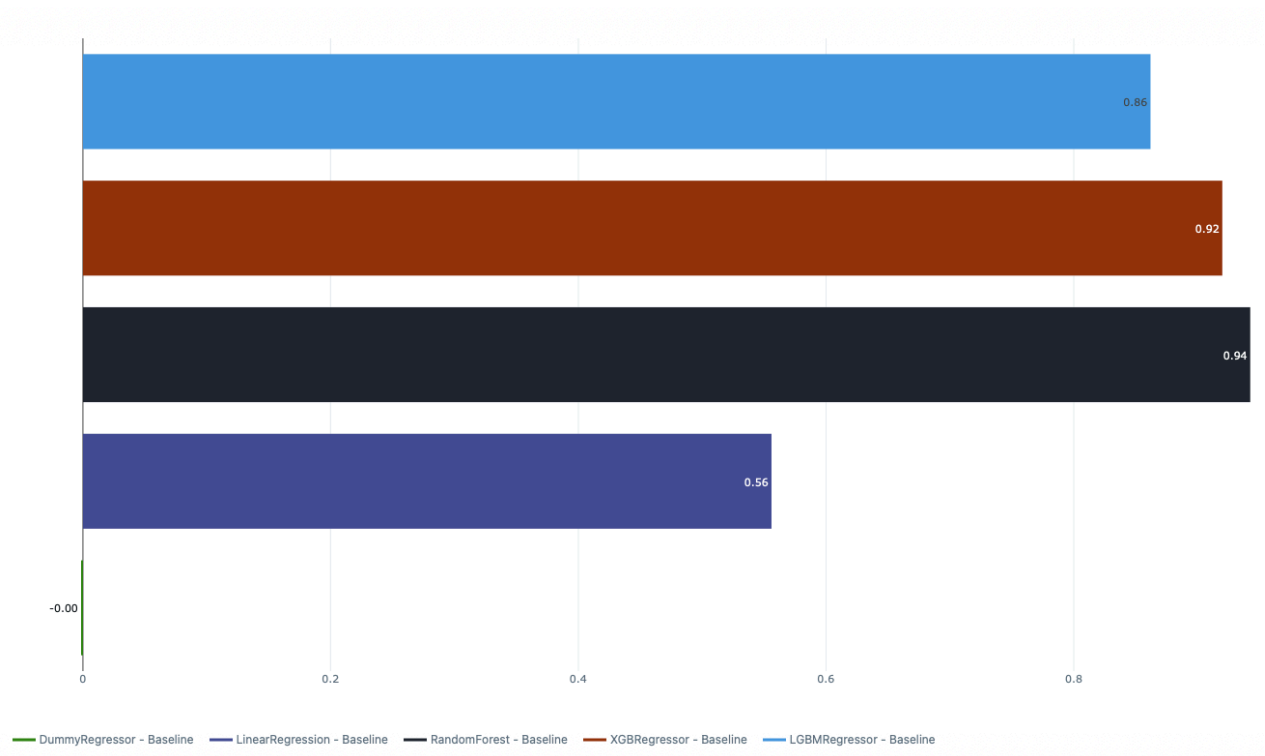
- LinearRegression :

```
=== Résultats métriques ===  
CV RMSE : 4840.4290 (± 55.16)  
CV MAPE : 0.9546 (± 0.0388)  
CV R2 : 0.5943 (± 0.0143)  
Test RMSE : 4805.9227  
Test R2 : 0.5559  
Test MAPE : 0.9859  
Test economic_error_usd_ha : 1033.3629
```

Comparaison sur tous les modèles

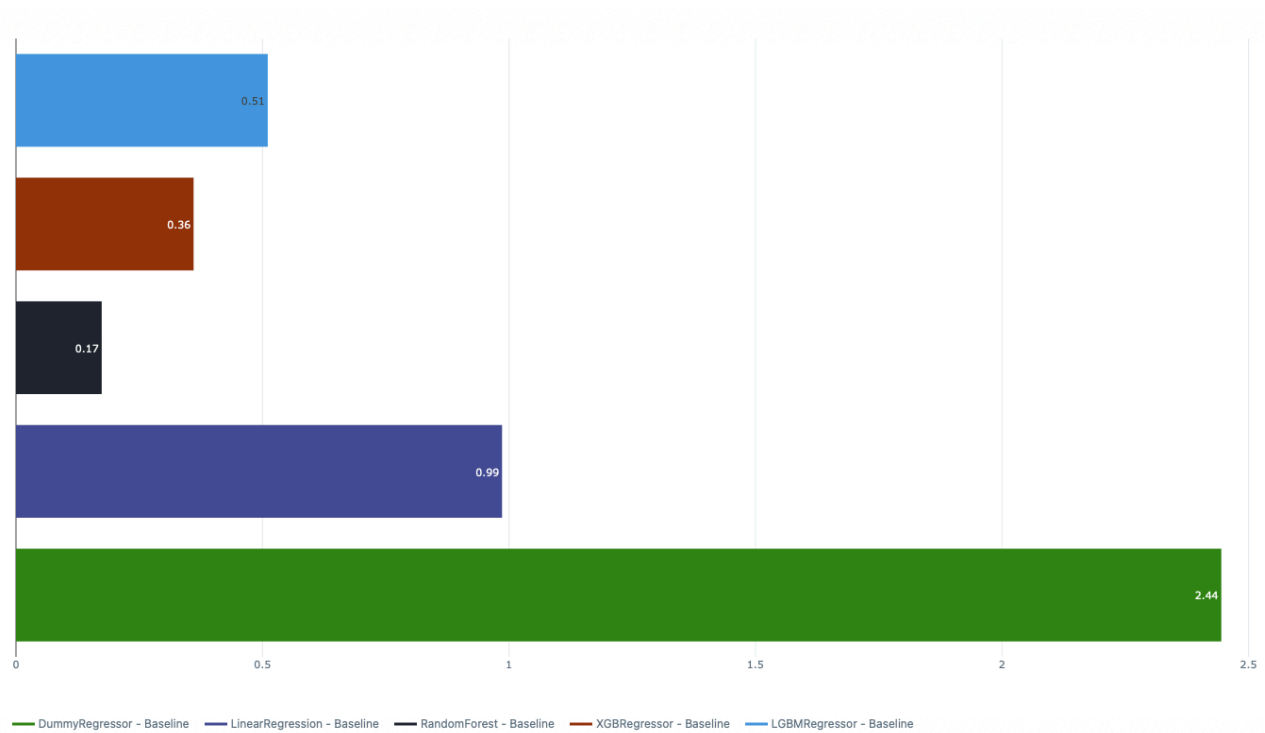
- Résultats des tests du R2

test_r2



- Résultats des tests du MAPE

test_mape



Les modèles d'arbres obtiennent des performances nettement supérieures aux modèles de référence.

Par exemple le RandomForest a "seulement" 17% d'erreur en valeur absolue, en comparaison d'une erreur sur 2 pour le LGBMRegressor.

Petit focus sur ces 2 modèles

RandomForestRegressor

```
=== Résultats métriques ===  
CV RMSE : 1893.0226 (± 45.34)  
CV MAPE : 0.2091 (± 0.0098)  
CV R2 : 0.9378 (± 0.0051)  
Test RMSE : 1729.6763  
Test R2 : 0.9425  
Test MAPE : 0.1739  
Test economic_error_usd_ha : 273.6834
```

- Sur la validation croisée on voit que l'écart-type entre les différents folds est petit, les résultats restent stables.
- 17% d'erreur (MAPE) sur le test. Cela peut sembler haut mais il faut prendre en compte le contexte métier ici où on sait que les rendements dépendent de beaucoup de facteurs et que les prédictions sont à ajuster avec sa connaissance terrain.

XGBRegressor

```

=== Résultats métriques ===
CV RMSE : 2152.0037 (± 71.11)
CV MAPE : 0.3938 (± 0.0186)
CV R2 : 0.9195 (± 0.0082)
Test RMSE : 2040.8158
Test R2 : 0.9199
Test MAPE : 0.3603
Test economic_error_usd_ha : 406.6547

```

- Même si les résultats ne sont pas mauvais ils restent bien en-dessous sur toutes les métriques.

Analyse du MAPE par type de culture et du economic_error_usd_ha sur notre meilleur modèle

- MAPE

Compare by	● RandomForest - Baseline
test_mape	0.17
test_mape_cassav	0.16
test_mape_maize	0.17
test_mape_plantai	0.18
test_mape_potato	0.14
test_mape_rice	0.13
test_mape_sorghu	0.22
test_mape_soybea	0.20
test_mape_sweet	0.17
test_mape_wheat	0.16
test_mape_yams	0.29

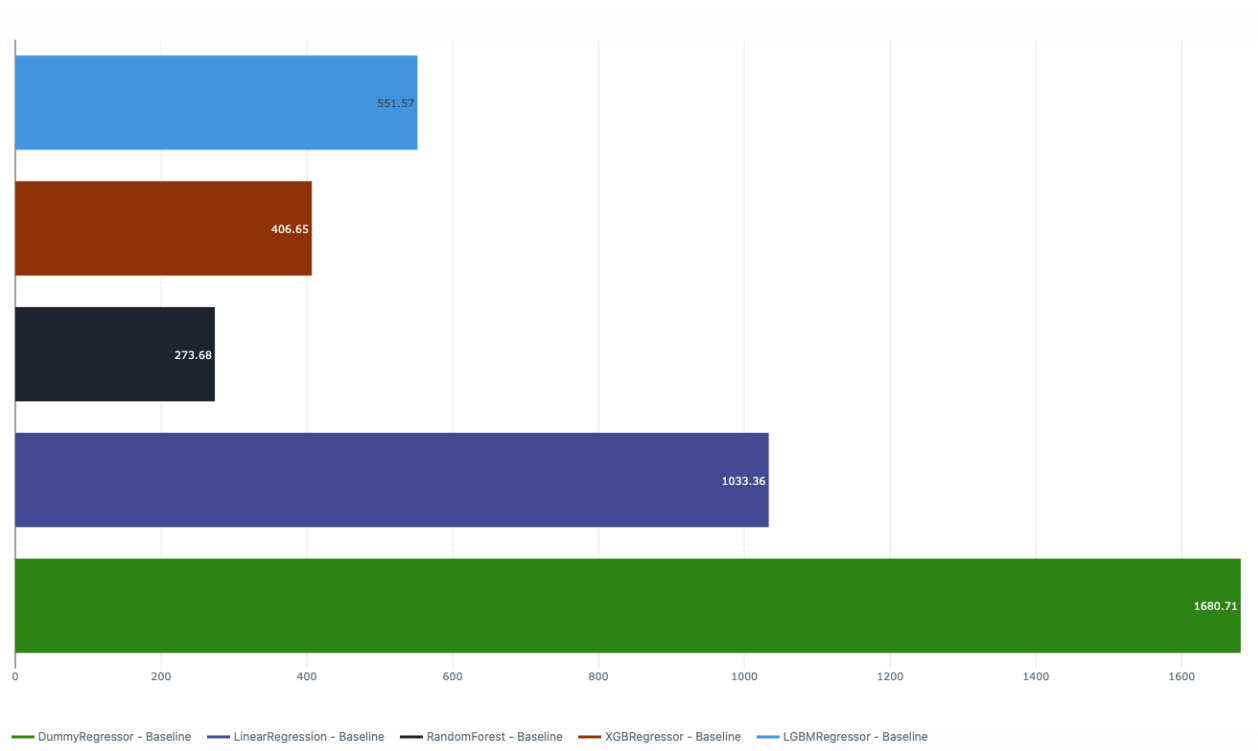
Le RandomForest affiche une excellente précision globale, mais l'analyse granulaire par culture via MLflow révèle des disparités intéressantes. Par exemple, le riz atteint une erreur de seulement 13%, tandis que l'igname monte à 29%. Cette différence s'explique par la plus grande variabilité biologique de certaines racines et potentiellement par un volume de données plus restreint sur ces catégories.

- Au-delà des métriques de modélisation classiques, une métrique métier a été introduite : `economic_error_usd_ha`.

Calcul de cette métrique :

- Calcul de l'erreur physique :
 - On calcule d'abord l'écart absolu entre la réalité et la prédiction en hectogrammes.
- Conversion en tonnes :
 - Comme les prix du marché mondial sont fixés à la tonne, on convertit l'erreur
- Valorisation monétaire :
 - On multiplie cette erreur par le prix spécifique de la culture fourni par la FAO (en USD/tonne)

economic_error_usd_ha



Passer d'une estimation 'au doigt mouillé' (Dummy) au meilleur modèle permet de sécuriser en moyenne plus de 1000 USD par hectare en précision de trésorerie.

- Pour le meilleur modèle, cette erreur est estimée à **273,7 USD/ha**. Le coût n'est pas négligeable selon la taille des champs mais cela reste une estimation et elle est la plus optimisée pour le moment.
- C'est très utile pour un exploitant agricole, parce qu'on traduit l'erreur de prédiction en impact financier moyen par hectare. Donc on peut répondre non seulement à "le modèle est-il précis ?", mais surtout à :
 - combien coûte une mauvaise estimation de rendement
 - si une recommandation est économiquement exploitable
- Le Random Forest présente le meilleur compromis entre précision, stabilité et interprétabilité.

Comparaison des performances des modèles sur le dataset brut et le dataset enrichi

Compare by	● RandomForest - Baseline - fichier enrichi	● RandomForest - Baseline
test_mae_kg	856.28	796.49 -59.80 ↓
test_mape	0.20	0.17 -0.02 ↓
test_mape_cassava	0.18	0.16 -0.02 ↓
test_mape_maize	0.19	0.17 -0.02 ↓
test_mape_plantains and others	0.28	0.18 -0.11 ↓
test_mape_potatoes	0.15	0.14 -0.01 ↓
test_mape_rice	0.14	0.13 -0.02 ↓
test_mape_sorghum	0.25	0.22 -0.02 ↓
test_mape_soybean	0.21	0.20 -0.01 ↓
test_mape_sweet potatoes	0.19	0.17 -0.02 ↓
test_mape_wheat	0.18	0.16 -0.01 ↓
test_mape_yams	0.37	0.29 -0.08 ↓
test_r2	0.94	0.94 +0.00 ↑
test_r2_cassava	0.86	0.87 +0.01 ↑
test_r2_maize	0.92	0.92 +0.00 ↑
test_r2_plantains and others	0.88	0.85 -0.03 ↓
test_r2_potatoes	0.92	0.92 +0.00 ↑
test_r2_rice	0.89	0.90 +0.01 ↑
test_r2_sorghum	0.87	0.89 +0.02 ↑
test_r2_soybean	0.77	0.77 +0.00 ↑
test_r2_sweet potatoes	0.89	0.91 +0.02 ↑
test_r2_wheat	0.91	0.92 +0.01 ↑

L'objectif était d'évaluer si cet enrichissement permettait d'améliorer la capacité de généralisation des modèles.

Les résultats obtenus montrent toutefois que cet enrichissement n'apporte pas de gain mesurable sur les performances, et peut même légèrement dégrader certaines métriques.

Ce résultat suggère que :

- soit les variables ajoutées ne contiennent pas d'information suffisamment discriminante,
- soit leur caractère indirect / approximatif limite leur contribution au modèle.

Le choix final s'est donc porté sur le **dataset brut**, afin de privilégier un pipeline plus simple et plus robuste.

Optimisation des hyperparamètres

Le **RandomForestRegressor** a été retenu comme modèle final pour trois raisons principales :

1. Il obtient les meilleures performances globales sur le dataset consolidé.
2. Il présente une bonne stabilité en validation croisée, avec une faible variabilité entre les folds.
3. Il offre un bon niveau d'interprétabilité via les analyses de feature importance et SHAP.

Ce choix répond donc à un compromis équilibré entre **performance prédictive**, **robustesse** et **lisibilité métier**.

Une phase d'optimisation par **GridSearchCV** a été menée sur ce dernier.

Les meilleurs hyperparamètres retenus sont les suivants (en comparaison avec la baseline):

Compare by	● RandomForest - GridSearchCV	● RandomForest - Baseline
▼ Parameters		
model__max_depth	25	None
model__max_features	0.7	1.0
model__n_estimators	200	100
verbose	0	False

Les gains observés après optimisation restent toutefois limités. Ce comportement suggère que le modèle se situe déjà proche de son optimum sur les données disponibles. En conséquence, les marges de progression futures semblent davantage liées à l'enrichissement des données qu'à une optimisation algorithmique supplémentaire.

Analyse du R2 et de la métrique économique pour ce modèle par type de culture :

- R2 :

Metric	Value
test_r2_cassava	0.8675881711244008
test_r2_maize	0.9227826744503265
test_r2_plantains and others	0.8460366641474585
test_r2_potatoes	0.9239807626911898
test_r2_rice	0.8988713711441583
test_r2_sorghum	0.8779277304688566
test_r2_soybean	0.7840899527913471
test_r2_sweet potatoes	0.9129412511673476
test_r2_wheat	0.9249098236718237
test_r2_yams	0.662231628581458

- Résultats :
 - Le modèle RandomForest s'impose comme une solution robuste avec un R2 global de 0.94, capturant une bonne partie des données.
 - Cependant, l'analyse par culture révèle une logique métier profonde, les cultures à haut rendement industriel, comme la pomme de terre ou le blé, affichent des scores d'excellence (R2>0.92 et MAPE basse), car elles répondent de manière prévisible aux intrants et au climat.
 - À l'inverse, des cultures comme l'igname présentent une complexité supérieure (R2 de 0.66 et MAPE de 0.29), soulignant une sensibilité à des facteurs non capturés.
- Métrique économique :

Metric	Value
test_economic_error_usd_ha_sorghum	71.33998066040844
test_economic_error_usd_ha_wheat	71.89900684899797
test_economic_error_usd_ha_soybean	88.31830875922314
test_economic_error_usd_ha_rice	123.23632012957917
test_economic_error_usd_ha_maize	137.09343474307082
test_economic_error_usd_ha_sweet potatoes	278.97992061218355
test_economic_error_usd_ha_cassava	316.5750518042259
test_economic_error_usd_ha_plantains and o...	325.1024254551195
test_economic_error_usd_ha_potatoes	547.6184288716083
test_economic_error_usd_ha_yams	1244.7143600707957

- Cultures à fort volume (ex: Potatoes) : Même si l'erreur en kg est élevée, le prix à la tonne est modéré (330 USD). L'impact financier est stable.
- Cultures à forte valeur (ex: igname) : À 890 USD/tonne, la moindre petite erreur de prédiction coûte très cher.
- L'introduction de l'Economic Error permet de traduire ces écarts en enjeux financiers réels.
- Prenons l'exemple de l'igname :
- Il est un point de vigilance économique. C'est la culture où l'erreur coûte le plus cher. Comme le prix à la tonne est très élevé, chaque tonne mal prédite par le modèle représente une perte de visibilité financière énorme (890 USD/t). C'est pour cela que même si le modèle est globalement bon, c'est sur ce genre de valeur que nous devons concentrer nos efforts pour minimiser cette 'Economic Error' et sécuriser le revenu des producteurs.

Interprétabilité du modèle feature importance et SHAP values

Afin de mieux comprendre les mécanismes de prédiction du modèle retenu, deux approches complémentaires d'interprétabilité ont été utilisées :

- la feature importance native du Random Forest
- les SHAP values, qui permettent d'analyser la contribution réelle de chaque variable aux prédictions.

Ces deux lectures ne mesurent pas exactement la même chose :

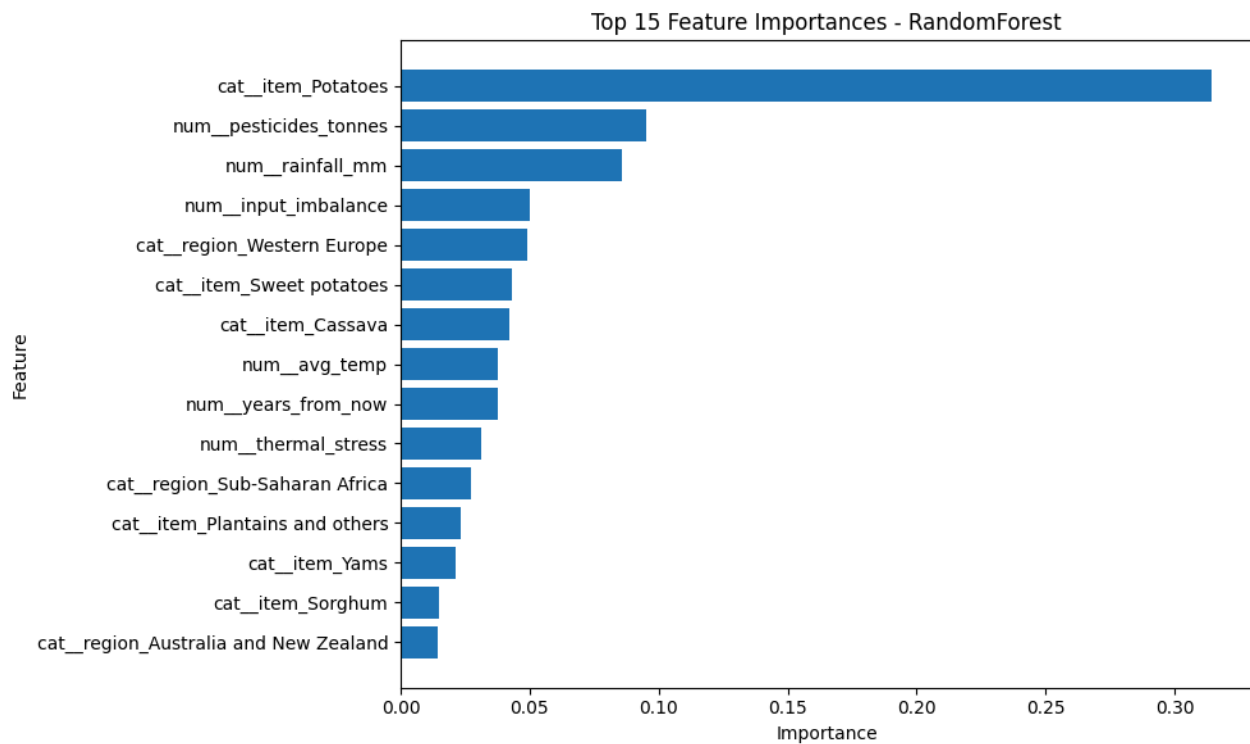
- la feature importance indique quelles variables sont le plus utilisées par le modèle pour réduire l'erreur lors des splits
- les SHAP values montrent dans quel sens et avec quelle intensité chaque variable fait varier une prédiction.

L'intérêt de croiser les deux approches est de distinguer :

- les variables structurellement importantes pour le modèle
- les variables qui ont l'impact le plus fort sur les prédictions individuelles.

1. Feature importance globale du Random Forest

- Top 15 des variables les plus importantes :



La variable la plus importante est de très loin item_Potatoes, avec un poids supérieur à 31 %. Cela signifie que le fait d'être sur la culture Potatoes structure fortement les décisions du modèle. Ce résultat est cohérent avec l'analyse exploratoire, qui montrait déjà que certaines cultures, notamment les pommes de terre, présentaient des niveaux de rendement beaucoup plus élevés que les autres.

On observe ensuite un second bloc de variables très influentes :

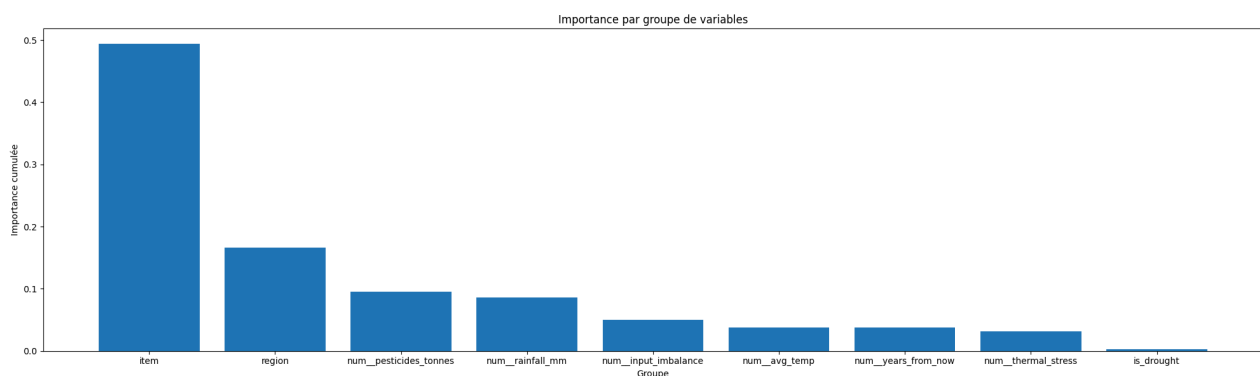
- pesticides_tonnes
- rainfall_mm
- input_imbalance

Ce groupe montre que le modèle ne repose pas uniquement sur le type de culture : il intègre aussi fortement les conditions agro-climatiques, les intrants.

Enfin, plusieurs variables de région apparaissent dans le top 15, ce qui confirme que le contexte géographique joue un rôle significatif dans l'estimation des rendements.

2. Importance par familles de variables

Pour rendre l'analyse plus lisible métier, les variables ont été regroupées en grandes familles.



- Le modèle s'appuie d'abord sur le type de culture, qui représente près de 49,4 % de l'importance totale. C'est le signal principal.
- La région pèse environ 16,6 %, ce qui confirme qu'un même type de culture ne se comporte pas de la même manière selon le contexte géographique.

- Les variables climatiques et techniques représentent également une grande importance. Cela montre que la prédiction ne dépend pas uniquement du choix de culture, mais aussi des conditions de production : pluie, température, pression d'intrants et déséquilibres entre eau et pesticides.
- Cette répartition est cohérente d'un point de vue métier :
 - la culture fixe un potentiel de rendement de base
 - le climat et les intrants modulent ce potentiel
 - la zone géographique ajoute un effet structurel supplémentaire.

3. Analyse des SHAP values

L'analyse SHAP affine cette lecture en montrant non seulement quelles variables comptent le plus, mais aussi dans quel sens elles influencent la prédiction.

Variables les plus influentes selon SHAP Les variables ressortant le plus fortement en contribution moyenne absolue sont :

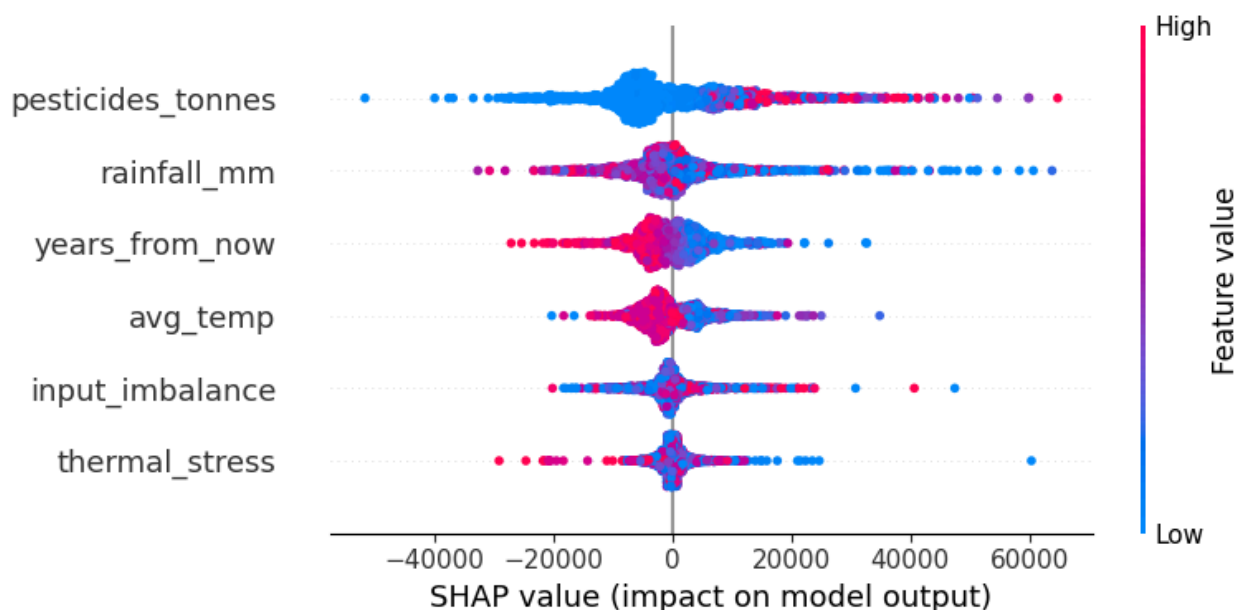


- Les SHAP values confirment que certaines cultures déplacent fortement la prédiction :
 - item_Potatoes a l'effet positif le plus fort du modèle

- item_Sweet potatoes, item_Cassava, item_Yams et item_Plantains and others ont également des contributions positives marquées
- à l'inverse, des cultures comme item_Soybean, item_Sorghum ou item_Wheat apparaissent davantage associées à des contributions négatives ou plus faibles.
- Le modèle apprend qu'à conditions comparables, certaines cultures sont associées à des niveaux de rendement structurellement plus élevés que d'autres.
- Les SHAP values montrent aussi des effets géographiques différenciés :
 - region_Sub-Saharan Africa contribue fréquemment négativement aux prédictions
 - region_Western Europe, region_Western Asia, region_Northern Europe et region_Eastern Asia ont plutôt des contributions positives.
- Effet du climat et de l'intrant. Plusieurs variables numériques ont un rôle important :
 - pesticides_tonnes : résultat plus contrasté avec les contributions positives et négatives plus concentrées
 - rainfall_mm : son effet est important mais dépend du contexte également
 - avg_temp : la température moyenne influence bien la prédiction, mais avec une intensité plus modérée que la pluie ou les pesticides
 - thermal_stress : un stress thermique élevé a plutôt un effet négatif
 - years_from_now : les valeurs élevées de cette variable, correspondant aux années les plus anciennes, ont tendance à tirer la prédiction vers le bas, tandis que les années plus récentes contribuent davantage positivement. Le modèle capte une tendance dans le temps avec l'idée d'une amélioration progressive des rendements au fil du temps, possiblement liée à l'évolution des pratiques, des intrants, des techniques ou des variétés.

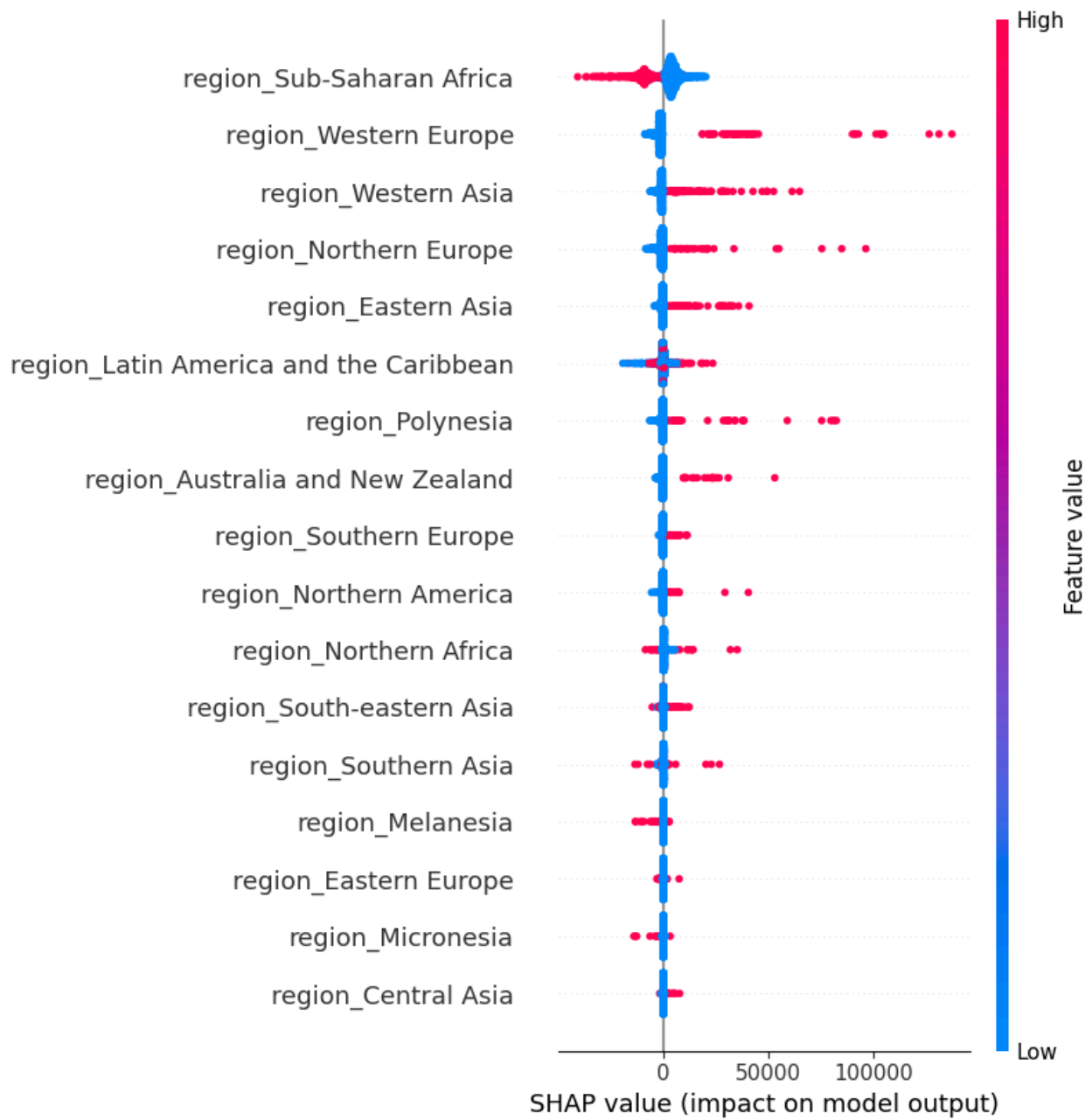
Regardons la contribution des SHAP Values par grande catégorie

- Sur nos variables numériques :

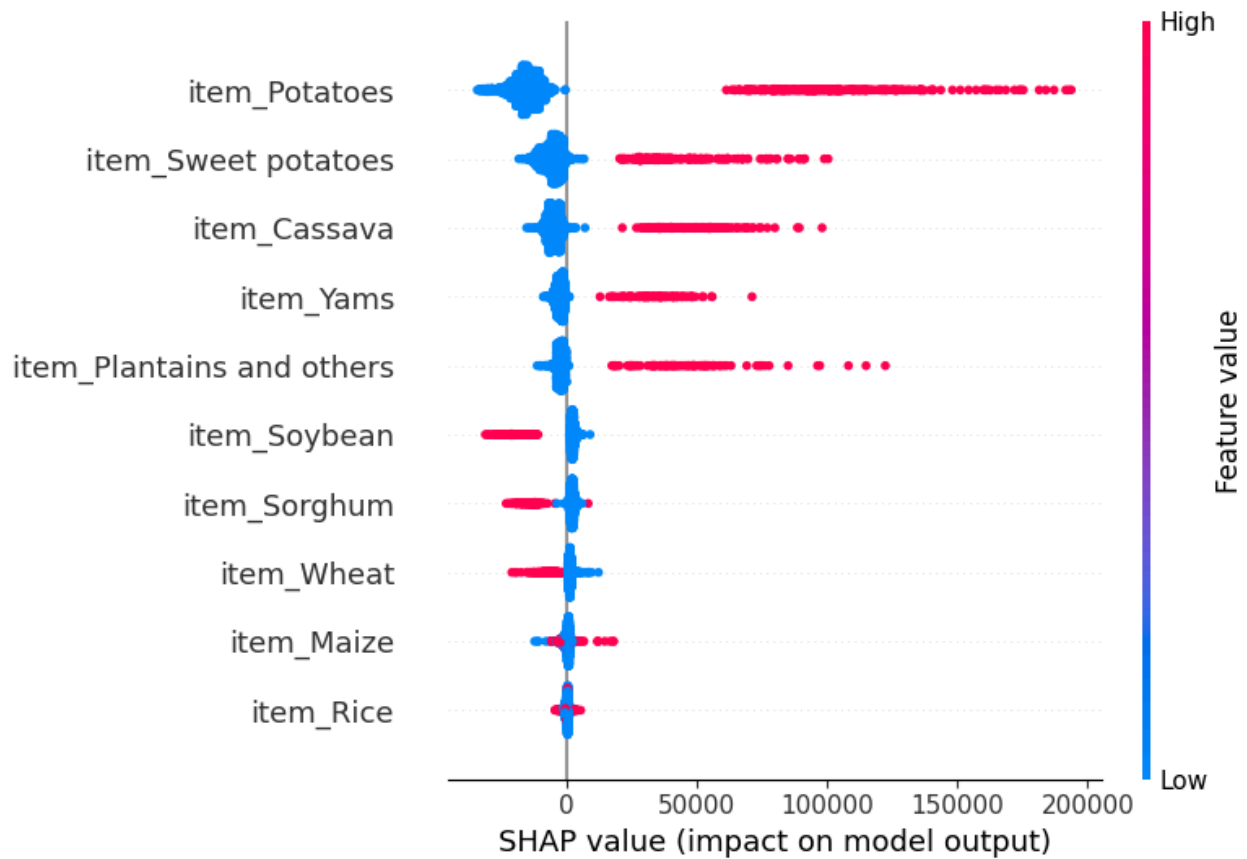


- Résultats :
 - Ce graphique montre par exemple que plus la valeur des pesticides ou des précipitations augmente (points roses), plus la prédiction de rendement est poussée vers le haut. Il permet de valider que le modèle réagit de manière logique aux variations environnementales et techniques.
 - Mais on voit une forte concentration pour ces variables vers le milieu, le contexte joue beaucoup sur la contribution de ces variables
- Sur les régions et les types de culture

Région



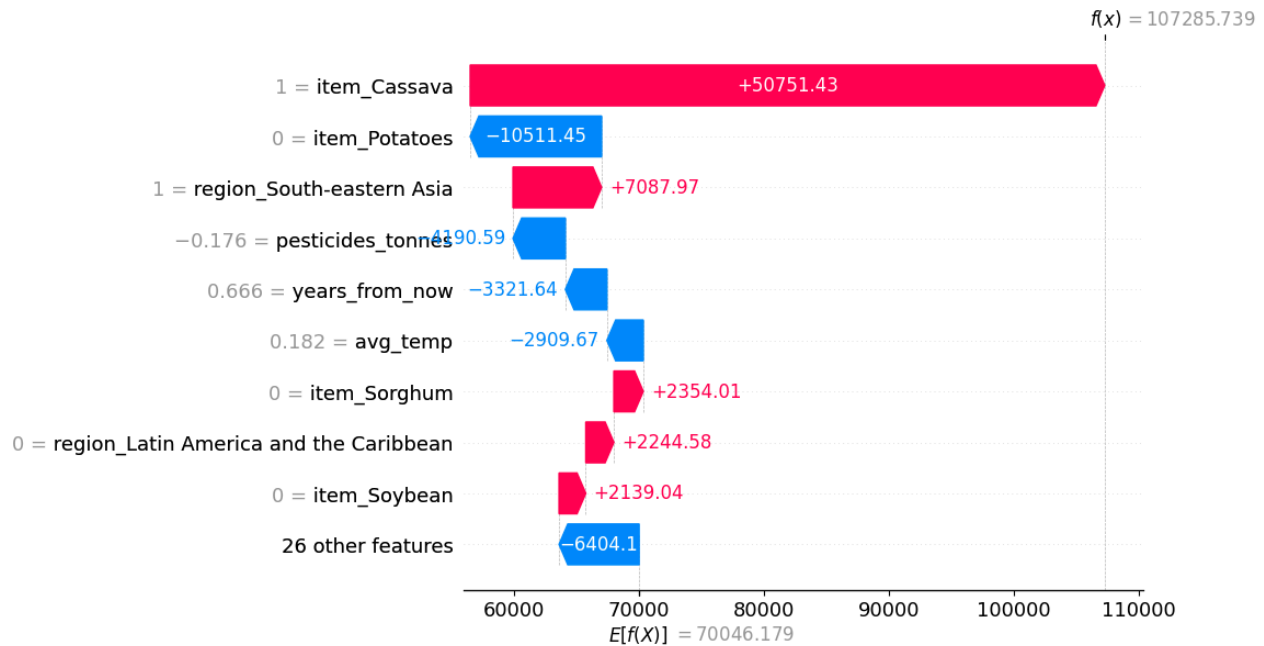
Type de culture



- Résultats :
 - En isolant les régions ou les cultures, les graphiques prouvent que le modèle a appris des spécificités géographiques (comme l'avantage structurel de certaines régions).
 - On voit sur le type de culture que certaines valeurs poussent largement le modèle vers le haut et d'autres clairement vers le bas.
 - Vers le haut : pomme de terre / igname / cassava / banane / patate douce
 - Vers le bas : soja / sorghum / blé
 - Cela montre que le modèle sait s'adapter au contexte local plutôt que d'appliquer une règle générique à toute la planète.

4. Analyse d'une prédiction avec waterfall plot

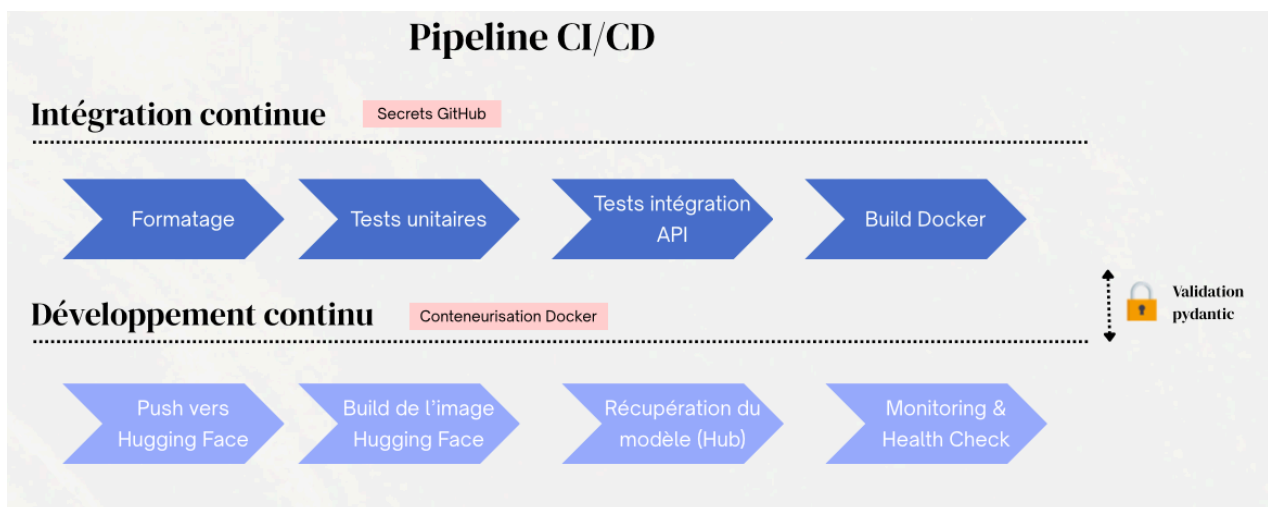
Cela part d'une prédiction moyenne du modèle sur le dataset d'entraînement, ensuite selon les contributions de certaines variables, le modèle va pousser la prédiction vers le haut ou vers le bas.



Dans ce cas précis on voit qu'une large partie de la contribution positive du model vient de la catégorie cassava et dans une moindre mesure la région sud-est asiatique.

Pipeline CI CD

Ce workflow GitHub Actions automatise les tests et le déploiement de l'application AgriTech Interface vers les Hugging Face Spaces.



Objectifs du workflow

Le but est de garantir que chaque modification poussée sur la branche principale est testée et immédiatement mise en ligne, assurant ainsi une intégration et un déploiement continu (CI/CD).

Déclencheurs (Triggers)

Le workflow s'active automatiquement selon deux scénarios :

- Push sur main : toute fusion ou push direct sur la branche principale.
- Manuel : peut être lancé manuellement depuis l'onglet "Actions" de GitHub.

Étapes du Job (build-and-deploy)

Le job s'exécute sur un environnement ubuntu-latest et suit les étapes suivantes :

1. Préparation de l'environnement

- Checkout : récupère le code source du dépôt. L'option `fetch-depth: 0` est utilisée pour obtenir tout l'historique (nécessaire pour certains outils de versioning ou déploiements git spécifiques).
- Setup Python : installe Python 3.12.
- Install Poetry : installe le gestionnaire de dépendances Poetry, configure le PATH et désactive la création de `venv` pour utiliser l'environnement global du runner.

2. Gestion des dépendances & Tests

- Install dependencies : installe les bibliothèques définies dans `pyproject.toml`.
- Run Unit Tests :
 - Installe `pytest` et `httpx`.
 - Définit le `PYTHONPATH` pour inclure le répertoire racine.
 - Exécute les tests situés dans le dossier `tests/`. Si les tests échouent, le déploiement est annulé.

3. Déploiement (Hugging Face Spaces)

Push to Hugging Face :

- Ajoute le dépôt distant Hugging Face via HTTPS en utilisant un token d'authentification.
- Force le push de la branche `main` locale vers le Space Hugging Face.

Détection automatique du docker par Hugging Face Spaces :

- il voit le `Dockerfile`
- il va lire et reconstruite chaque ligne du `Dockerfile`
- Une fois l'image construite -> interface Streamlit dans le Space

Configuration Requisite (Secrets)

Pour que ce workflow fonctionne, vous devez configurer le secret suivant dans les paramètres de votre dépôt GitHub (Settings > Secrets and variables > Actions)

Notes Techniques

- Hébergement cible : huggingface.co/spaces/FLORIANSC/agritech-interface
- Forçage du Push : le workflow utilise `git push --force`. Cela signifie que l'historique sur Hugging Face sera écrasé par celui de GitHub à chaque déploiement.
- Optimisation : les dépendances sont installées sans interaction (`--no-interaction`) pour éviter que le runner ne reste bloqué sur une question.

Tests et validation des données

La fiabilité du système repose sur deux piliers : la validation stricte des entrées avec Pydantic et une suite de tests automatisés intégrée au cycle CI/CD.

1. Validation de données avec Pydantic

- Pour garantir l'intégrité des prédictions, l'API utilise des schémas Pydantic (`InputPrediction` et `InputRecommendation`).
- Validation d'entrée : Chaque requête est vérifiée en temps réel. Si un utilisateur envoie une chaîne de caractères au lieu d'un nombre pour la température, l'API rejette la requête proprement avec une erreur `422 Unprocessable Entity`.
- Documentation auto-générée : Grâce aux `Field` et `json_schema_extra`.

2. Suite de tests (Pytest)

Le projet inclut une batterie de tests unitaires et d'intégration pour assurer la stabilité du code :

- Tests d'API (Integration Tests) :
 - `test_predict_endpoint` & `test_recommend_ok` : vérifient que les prédictions et recommandations renvoient des résultats cohérents et un code HTTP 200.

- `test_404_handler` & `test_predict_validation_error` : assurent que l'API gère les erreurs
- Tests de nettoyage (Unit Tests) :
 - `test_preparation_inference_columns` : Vérifie que le script de nettoyage transforme correctement les données brutes et génère bien les colonnes calculées. Pour que le pipeline de données est identique entre l'entraînement et l'inférence.
 - Intégrité CI/CD : un test de base (`test_ci_cd_pipeline`) valide le bon fonctionnement du moteur de test dans le pipeline GitHub Actions.
- 3. Couverture de code (Pytest-Cov)
- Le projet intègre `pytest-cov` dans le pipeline CI/CD. À chaque mise à jour :
 - Les tests sont exécutés.
 - Un rapport de couverture (`term-missing`) est généré, identifiant précisément les lignes de code non testées.
 - Le déploiement vers Hugging Face est bloqué si les tests échouent, garantissant une version de production toujours fonctionnelle.

Conclusion

Ce projet démontre qu'un pipeline MLOps structuré permet de transformer des données agricoles disparates en un outil d'aide à la décision performant. Le modèle Random Forest, avec un R2 global de 0.94, ne se contente pas de prédire des rendements, il capture les dynamiques complexes entre climat, géographie et techniques agricoles.

En traduisant l'erreur statistique en un risque financier en USD/ha, nous sortons du cadre purement mathématique pour parler le langage de l'agriculteur. Le projet prouve ainsi sa valeur ajoutée :

- réduire l'incertitude de trésorerie de plus de 75 % par rapport à une approche statistique classique (Baseline)
- sécurisant particulièrement les cultures à haute valeur ajoutée comme l'igname, où l'enjeu financier par tonne est le plus critique.
- grâce à l'interprétabilité SHAP, on est capable de justifier chaque prédiction.

Perspectives

Pour aller plus loin nous aurons besoin de pouvoir enrichir nos données de manière plus précise. Actuellement le dataset enrichi n'est pas utilisé car les variables rajoutées n'apportent rien au modèle. Nous avons donc besoin :

- Enrichissement du feature engineering :
 - Intégrer la composition des sols (pH, nutriments) pour affiner les prédictions certaines cultures qui affichent actuellement une variance plus complexe à expliquer.
 - Coupler l'API avec des données de télédétection pour monitorer l'état des cultures en temps réel et ajuster les prédictions en cours de saison.
 - Coupler également avec une API pour avoir des données de pluviométrie en direct
- Optimisation de l'infrastructure MLOps :
 - Mettre en place des "Model Drift Alarms" dans MLflow pour déclencher un réentraînement automatique du modèle dès que les conditions climatiques mondiales dérivent trop des données historiques.
- Expansion de la valeur métier :
 - Intégration des prix locaux : connecter l'API à des flux de prix régionaux en temps réel pour affiner la métrique économique en fonction des marchés locaux plutôt que des moyennes mondiales de la FAO.